The data items on this sheet (ignoring headings) can be listed as a set of attributes:

(BookingID, VenueName, VenueAddress1, VenueAddress2, Date,

(BandName, NumberOfMembers, Headlining))

The list is put inside brackets because we are starting a process of table design. The extra set of brackets around BandName, NumberOfMembers, Headlining is because they represent a **repeating group**. If there is a repeating group, the attributes cannot sensibly be put into one relational table. A table must have single rows and atomic attribute values so the only possibility would be to include tuples such as those shown in Table 11.05. There is now data redundancy here with the duplication of the BookingID, venue data and the date.

| Booking ID | Venue Name | Venue Address1 | Venue Address2 | Date | Band Name | Number Of Members | Headlining |
|---|---|---|---|---|---|---|---|
| 2016/023 | Cambridge International Theatre | Camside | CA1 | 23.06.2016 | Computer Kidz | 5 | Y |
| 2016/023 | Cambridge International Theatre | Camside | CA1 | 23.06.2016 | ITWizz | 3 | N |
| 2016/023 | Cambridge International Theatre | Camside | CA1 | 23.06.2016 | DeadlyDuo | 2 | N |

Table 11.05 Data stored in an unnormalised table



Step 1: Conversion to first normal form (1NF)

The conversion to first normal form (1NF) requires splitting the data into two groups. At this stage we represent the data as table definitions. Therefore, we have to choose table names and identify a primary key for each table. One table contains the non-repeating group attributes, the other the repeating group attributes. For the first table a sensible design is:

Booking(BookingID, VenueName, VenueAddress1, VenueAddress2, Date)

The table with the repeating group is not so straightforward. It needs a compound primary key and a foreign key to give a reference to the first table. The sensible design is:

Band-Booking(BandName, BookingID(fk), NumberOfMembers, Headlining)

Again, the primary key is underlined but also the foreign key has been identified, with (fk). Because the repeating groups have been moved to a second table, these two tables could be implemented with no data redundancy in either. This is one aspect of 1NF. Also, we can say that for each table the attributes are dependent on the primary key.

Step 2: Conversion to second normal form (2NF)

For conversion to second normal form (2NF), the process is to examine each non-key attribute and ask if it is dependent on both parts of the compound key. Any attributes that are dependent on only one of the attributes in the compound key must be moved out into a new table. In this case, NumberOfMembers is only dependent on BandName. In 2NF there are now three table definitions:

Booking(BookingID, VenueName, VenueAddress1, VenueAddress2, Date)

Band-Booking(BandName(fk), BookingID(fk), Headlining)

Band(BandName, NumberOfMembers)

*(Continued)*

Note that the Booking table is unchanged from 1NF. The Booking table is automatically in 2NF; only tables with repeating group attributes have to be converted. The Band-Booking table now has two foreign keys to provide reference to data in the other two tables. The characteristics of a table in 2NF is that it either has a single primary key or it has a compound primary key with any non-key attribute dependent on both components.

Step 3: Conversion to third normal form (3NF)

For conversion to third normal form (3NF) each table has to be examined to see if there are any non-key dependencies; that means we must look for any non-key attribute that is dependent on another non-key attribute. If there is, a new table must be defined.

In our example, VenueAddress1 and VenueAddress2 are dependent on VenueName. With the addition of the fourth table we have the following 3NF definitions:

Band(BandName, NumberOfMembers)

Band-Booking(BandName(fk), BookingID(fk), Headlining)

Booking(BookingID, Date, VenueName(fk))

Venue(VenueName, VenueAddress1, VenueAddress2)

Note that once again a new foreign key has been identified to keep a reference to data in the newly created table. These four table definitions match four of the entities in the logical E–R model for which the keys were identified in Table 11.04. This will not always happen. A logical E–R diagram will describe a 2NF set of entities but not necessarily a 3NF set.



KEY TERM

**Repeating group:** a set of attributes that have more than one set of values when the other attributes each have a single value

To summarise, if a set of tables are in 3NF it can be said that each non-key attribute is dependent on the key, the whole key and nothing but the key.

### Question 11.03

In Step 2 of Worked Example 11.02, why is the Headlining attribute not placed in the Band table?

**TASK 11.02**

Normalise the data shown in Figure 11.11.

| **Order no:** 07845 | | | | **Date:** 25-06-2016 |
|---|---|---|---|---|
| **Customer no:** 056 | | **Customer name:** | CUP | |
| | | **Address:** | Cambridge square | Cambridge |
| **Sales rep no:** | 2 | **Sales Rep name:** | Dylan Stoddart | |

| **Product no** | **Description** | **Quantity** | **Price / unit** | **Total** |
|---|---|---|---|---|
| 327 | Inkjet cartridges | 24 | $30 | $720 |
| 563 | Laser toner | 5 | $25 | $125 |
| | | **Total Price** | | $835 |

Figure 11.11 An order form

# 11.06 The Database Management System (DBMS)

## The database approach

It is vital to understand that a database is not just a collection of data. A database is an implementation according to the rules of a theoretical model. The basic concept was proposed some 40 years ago by ANSI (American National Standards Institute) in its three-level model. The three levels are:

- the external level
- the conceptual level
- the internal level.

The architecture is illustrated in Figure 11.12 in the context of a database to be set up for our theatrical agency.

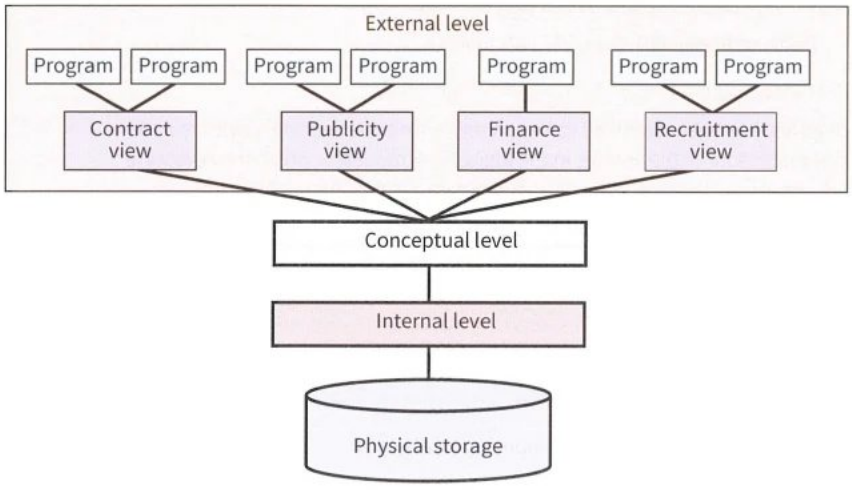



Figure 11.12 The ANSI three-level architecture for the theatrical agency database

The physical storage of the data is represented here as being on disk. The details of the storage (the internal schema) are known only at the internal level, the lowest level in the ANSI architecture. This is controlled by the **database management system (DBMS)** software.

The programmers who wrote this software are the only ones who know the structure for the storage of the data on disk. The software will accommodate any changes that might be needed in the storage medium.

At the next level, the conceptual level, there is a single universal view of the database. This is controlled by the **database administrator (DBA)** who has access to the DBMS. In the ANSI architecture the conceptual level has a conceptual schema describing the organisation of the data as perceived by a user or programmer. This may also be described as a logical schema.

At the external level there are individual user and programmer views. Each view has an external schema describing which parts of the database are accessible. A view can support a number of user programs.

An important aspect of the provision of views is that they can be used by the DBA as a mechanism for ensuring security. Individual users or groups of users can be given appropriate access rights to control what actions are allowed for that view. For example,

a user may be allowed to read data but not to amend data. Alternatively, there may only be access to a limited number of the tables in the database.

KEY TERMS

**Data management system (DBMS):** software that controls access to data in a database

**Database administrator (DBA):** a person who uses the DBMS to customise the database to suit user and programmer requirements

## The facilities provided by a DBMS

You need to remember that databases come in a variety of forms ranging from a simple system created for one individual through to the central database for some large organisation. Some of the facilities provided by a DBMS are only relevant for large organisations, when their use will be controlled by a DBA.

Whatever the size of the database, one option for its creation is to use the special-purpose language SQL which is discussed in the next section of this chapter. There are alternatives to SQL for most types of DBMS. The DBMS provides software tools through a **developer interface**. These allow for tables to be created and attributes to be defined together with their data types. In addition, the DBMS provides facilities for a programmer to develop a user interface. It also provides a **query processor** that allows a query to be created and processed. The **query** is the mechanism for extracting and manipulating data from the database. The other feature likely to be provided by the DBMS is the capability for creating a report to present formatted output. A programmer can incorporate access to queries and reports in the user interface.



KEY TERMS

**Developer interface:** gives access to software tools provided by a DBMS for creating tables

**Query processor:** software tools provided by a DBMS to allow creation and execution of a query

**Query:** used to select data from a database subject to defined conditions

## DBMS functions likely to be used by a DBA

The DBA is responsible for setting up the user and programmer views and for defining the appropriate, specific access rights.

An important feature of the DBMS is the data dictionary which is part of the database that is hidden from view from everyone except the DBA. It contains metadata about the data. This includes details of all the definitions of tables, attributes and so on but also of how the physical storage is organised.

There are a number of features that can improve performance. Of special note is the capability to create an **index** for a table. This is needed if the table contains a large number of attributes and a large number of tuples. An index is a secondary table that is associated with an attribute that has unique values. The index table contains the attribute values and pointers to the corresponding tuples in the original table. The index can be on the primary key or on a secondary key. Searching an index table is much quicker than searching the full table.

KEY TERM

**Index:** a small secondary table used for rapid searching which contains one attribute from the table being searched and pointers to the tuples in that table

The integrity of the data in the database is a key concern. One potential cause of problems occurs when a transaction is started but a system problem prevents its completion. The result would be a database in an undefined state. The DBMS should have a built-in feature that prevents this from happening. As with all systems, regular backup is a requirement. The DBA will be responsible for backup of the stored data.

**Discussion Point:**

How many of the above concepts are recognisable in your experience of using a database?

## 11.07 Structured Query Language (SQL)

SQL is the programming language provided by a DBMS to support all of the operations associated with a relational database. Even when a database package offers high-level software tools for user interaction, they create an implementation using SQL.

### Data Definition Language (DDL)

Data Definition Language (DDL) is the part of SQL provided for creating or altering tables. These commands only create the structure. They do not put any data into the database.

The following are some examples of DDL that could be used in creating the database designed in Worked example 11.02 for the theatrical agency:

```
CREATE DATABASE BandBooking;
CREATE TABLE Band (
    BandName varchar(25),
    NumberOfMembers integer);
ALTER TABLE Band ADD PRIMARY KEY (BandName);
ALTER TABLE Band-Booking ADD FOREIGN KEY (BandName REFERENCES
Band(BandName);
```



These examples illustrate a number of general points regarding the writing of SQL.

- The SQL consists of a sequence of commands.
- Each command is terminated by ;
- A command can occupy more than one line.
- There is no case sensitivity.
- There has been a decision made here to use upper case for the commands and lower case for table names, attribute names and datatypes.
- When a command contains a list of items these are separated by a comma.
- For the CREATE TABLE command this list is enclosed in parentheses

These examples show that once the database has been created, the tables can be created and the attributes defined. It is possible to define a primary key and a foreign key within the `CREATE TABLE` command but the `ALTER TABLE` command can be used as shown (it can also be used to add extra attributes).

When an attribute is defined, its data type must be specified. As with procedural languages there can be different data types or different names for data types depending on which DBMS is being used. One feature common to all databases is that the number of characters allowed for an attribute can be defined by including the number in brackets. In the above example `BandName varchar(25)` allows up to 25 characters for the band name.